INTRODUCING COCOON 2.1

author: Steven Noels
date: November 2003
venue: ApacheCon, Las Vegas
A gentle introduction to Cocoon 2.1

1. ABOUT THIS TALK

Two intro-type talks are scheduled this afternoon.

One focuses on vision and advocacy, one on getting the thing up and running (this one).

If you want the full story, stick around for the next talk as well.

60' is a tight fit, but we'll (try to) manage.

Please turn off your cell phones.

2. WHAT IS COCOON?

What it used to be: an XML-centric web publishing framework.

» XML processing pipelines
» multiple media publishing
» portals and content aggregation

What it is evolving into: a generic XML-centric web application development platform.

» JavaScript flowscript and webcontinuations
» Cocoon Forms aka Woody
» Blocks like mini-WARs adding comprehensive services to a webapp

3. THE COCOON COMMUNITY

Cocoon has recently moved into a top-level ASF project.

One incubatee: Lenya CMS.

60-ish commit karma, ±15 active peeps.

Approx. 1500 messages/month on both users and dev list.

Website, Wiki, the works.

Has its own community event: the Cocoon GetTogether.

4. RUNNING COCOON

Java Servlet (J2EE)

» runs on most Servlet containers
» Jetty & Tomcat, but (many) commercial ones as well
» provides ParanoidServlet with shielded classloader for nasty situations

CLI environment

» Cocoon crawler
» It's like having wget built-in.

Cocoon Bean

5. STARTING WITH COCOON

download or cvs co cocoon-2.1

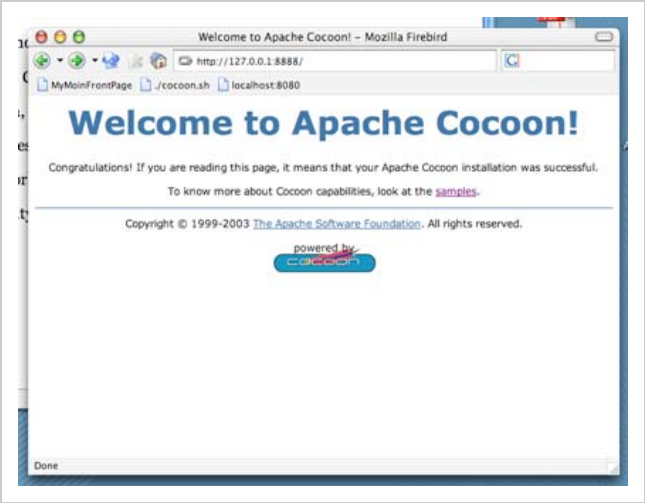
(unpack)

edit local.{build|blocks}.properties

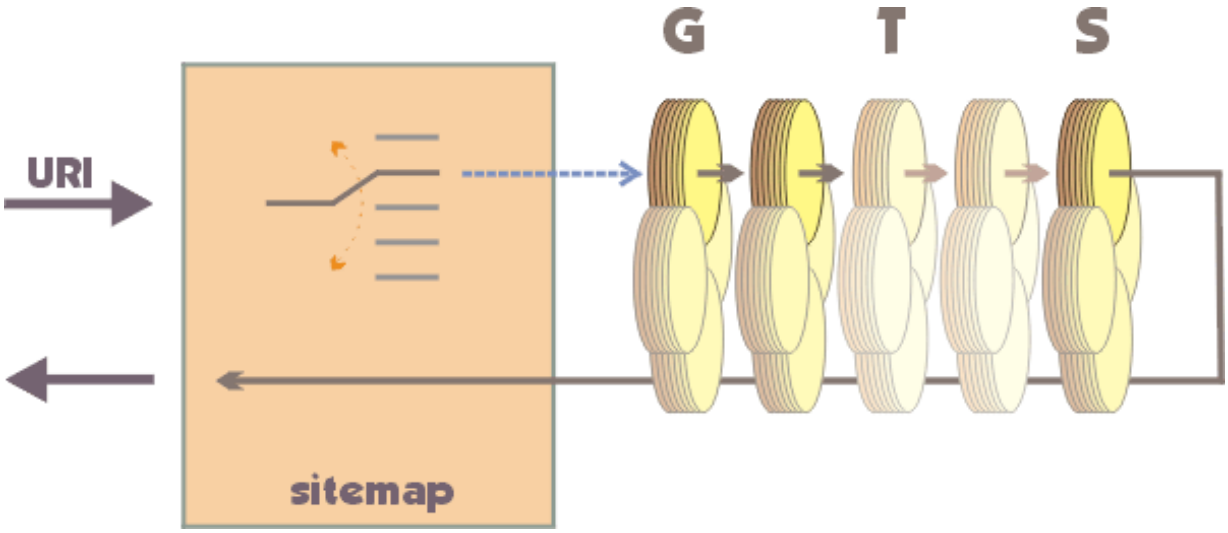
./build.sh webapp

./cocoon.sh servlet

done: http://127.0.0.1:8888/



6. COCOON FUNDAMENTALS



» A request is analysed by the sitemap against a set of matchers.
» A response pipeline is selected and constructed.
» A response is generated and sent back.

7. COCOON PIPELINES

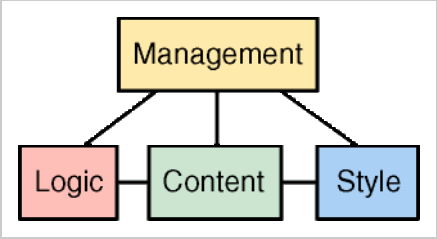
A Cocoon pipeline consists of several components, tied together using SAX events.

- » Generators: generate SAX events from a source
- » Transformers: apply a transformation onto incoming SAX events, forwarding outgoing SAX events
- » Serializers: sink incoming SAX stream into outgoing byte stream

Readers exist which simply read a byte stream from a source, set a mime type and bypass SAX machinery, sending directly to output (e.g. images).

8. THE COCOON SITEMAP

How does one expresses the binding between an incoming request and the responding pipeline?



The "Management" concern.

Matchers: describe a pattern against which an incoming request should be matched.

- » Wildcard or regex patterns
- » Request Parameters
- » Cookies
- » Session Attributes
- » ...

```
<map:match pattern="**/*.xhtml">
  ...
  [pipeline stuff goes here]
  ...
</map:match>
```

Matched pattern groups can be reused in the pipeline component configuration.

9. SITEMAP EXAMPLE

```
<map:match pattern="**/*.xhtml">
  <map:generate src="{1}/{2}.xml"/>
  <map:transform src="stylesheets/doc2xhtml.xsl">
    <map:parameter name="section" value="{1}"/>
  </map:transfom>
  <map:serialize type="xhtml"/>
</map:match>
<map:match pattern="**/*.pdf">
  <map:generate src="{1}/{2}.xml"/>
  <map:transform src="stylesheets/doc2fo.xsl"/>
  <map:serialize type="fo2pdf"/>
</map:match>
```

Serializer used: Apache FOP. Others: HSSF (POI), OpenOffice, Batik, iText, jfor ...

10. COCOON CONFIGURATION
WEB-INF/cocoon.xconf

(Global) component declaration and configuration (Avalon)

Pools (databases, components)

sitemap.xmap

Components: naming and default configuration

Resources: pipeline macros

Views: orthogonal to Matchers and pipelines, used for link gathering, crawling, semantic views...

Pipelines

map:mount for subsitemaps

FlowScript calls

Error-handling pipelines

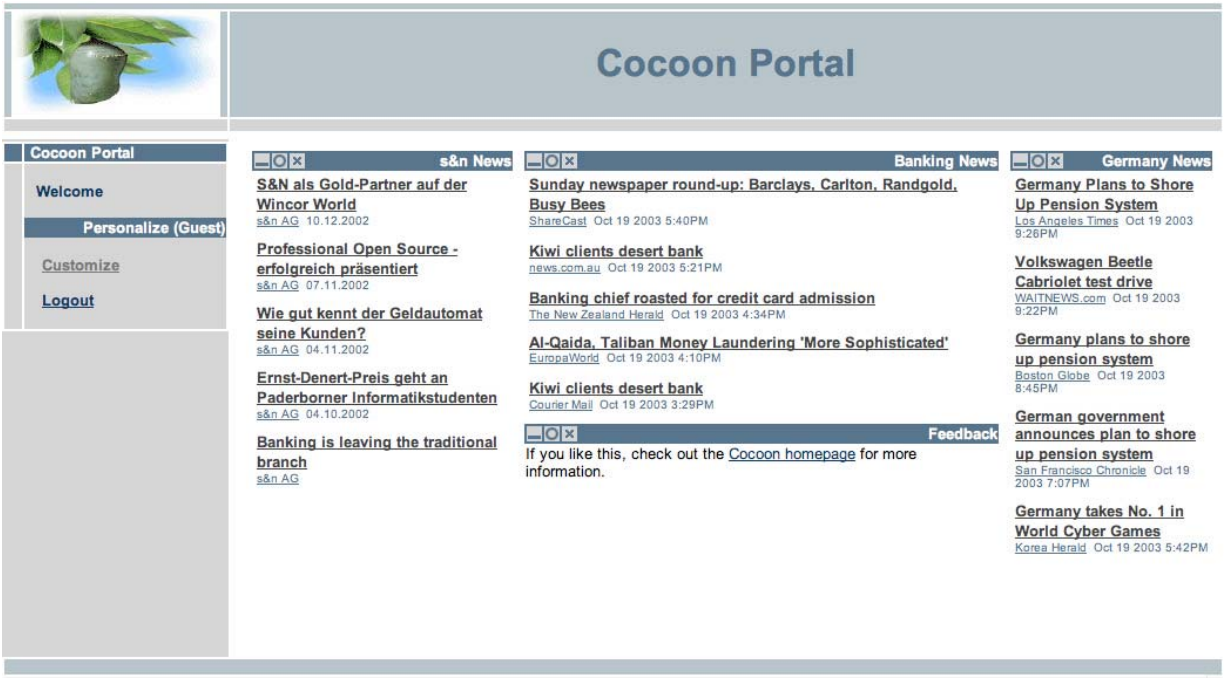
11. CONTENT AGGREGATION
Sitemap aggregation

```
<map:aggregate element="newspage">
  <map:part element="header" src="cocoon:/foo"/>
  <map:part element="content" src="cocoon:/bar"/>
  <map:part element="footer" src="cocoon:/baz"/>
</map:aggregate>
```

Aggregation transformers

» cinclude
» XInclude (with XPointer)

12. ANOTHER TAKE AT AGGREGATION: COCOON PORTAL FRAMEWORK



13. ADDING DYNAMISM TO THE MIX

- » Custom components
- » Actions
- » XSP
- » Flow: scripting Avalon components with JavaScript and web continuations

Typical evolution: XSP -> Actions/Custom components -> Flow + Avalon components



14. CUSTOM COCOON COMPONENTS

Avalon

Component life-cycle management APIs

Configuration mechanism (xconf)

ContentHandler interface

Create your own Source, Matcher, Transformer, etc ...

15. ACTIONS

Are (only) able to configure the setup of the processing pipeline.

Cannot access the SAX stream passed across the pipeline.

Able to make pipeline sections optional.

```
<map:match pattern="login">
  <map:act type="auth-login">
    <map:parameter name="handler" value="portalhandler"/>
    <map:parameter name="parameter_userid" value="{reques
    <map:parameter name="parameter_password" value="{requ
    <!-- authentication successful: -->
    <map:redirect-to uri="authentication-successful"/>
  </map:act>
  <!-- authentication failed: -->
  <map:generate src="auth_failed.xml"/>
  <map:transform src="tohtml.xsl"/>
  <map:serialize/>
</map:match>
```

16. XSP

eXtensible Server Pages

JSP reformulation using XML

mix of Java and XML

Are transformed (using XSL) into Generators

Logicsheets

```
<xsp:page language="java"
  xmlns:xsp="http://apache.org/xsp"
  xmlns:xsp-request="http://apache.org/xsp/request/2.0"
  xmlns:util="http://apache.org/xsp/util/2.0"
  xmlns:sendmail="http://apache.org/cocoon/sendmail/1.0">
  <email>
    <sendmail:send-mail>
      <sendmail:from>from@example.org</sendmail:from>
      <sendmail:to>to@example.com</sendmail:to>
      <sendmail:subject>Test Email from Cocoon</sendmail
      <!-- Modify the next line to point to your mail se
      <sendmail:smtphost>smtp.example.org</sendmail:smtpl
      <sendmail:body><xsp:expr>
        <util:get-source uri="cocoon://mail/payload"/>
        </xsp:expr></sendmail:body>
      <sendmail:on-success>
        <p>Email successfully sent.</p>
      </sendmail:on-success>
      <sendmail:on-error>
        <p style="color:red;">An error occurred: <sendma
      </sendmail:on-error>
    </sendmail:send-mail>
  </email>
</xsp:page>
```

17. FLOW

JavaScript with WebContinuations

See Stefano's presentation

Suggested template language: JXTemplateGenerator and -Transformer

» JXPath
» JEXL

18. COCOON COMPONENTS AND BLOCKS

Basic transformations

» TRAX / Xalan / Saxon
» XSLTC
» STX

Multi-device and -media

» FOP & iText, jfor
» Batik
» HSSF / POI (Excel)
» Deli

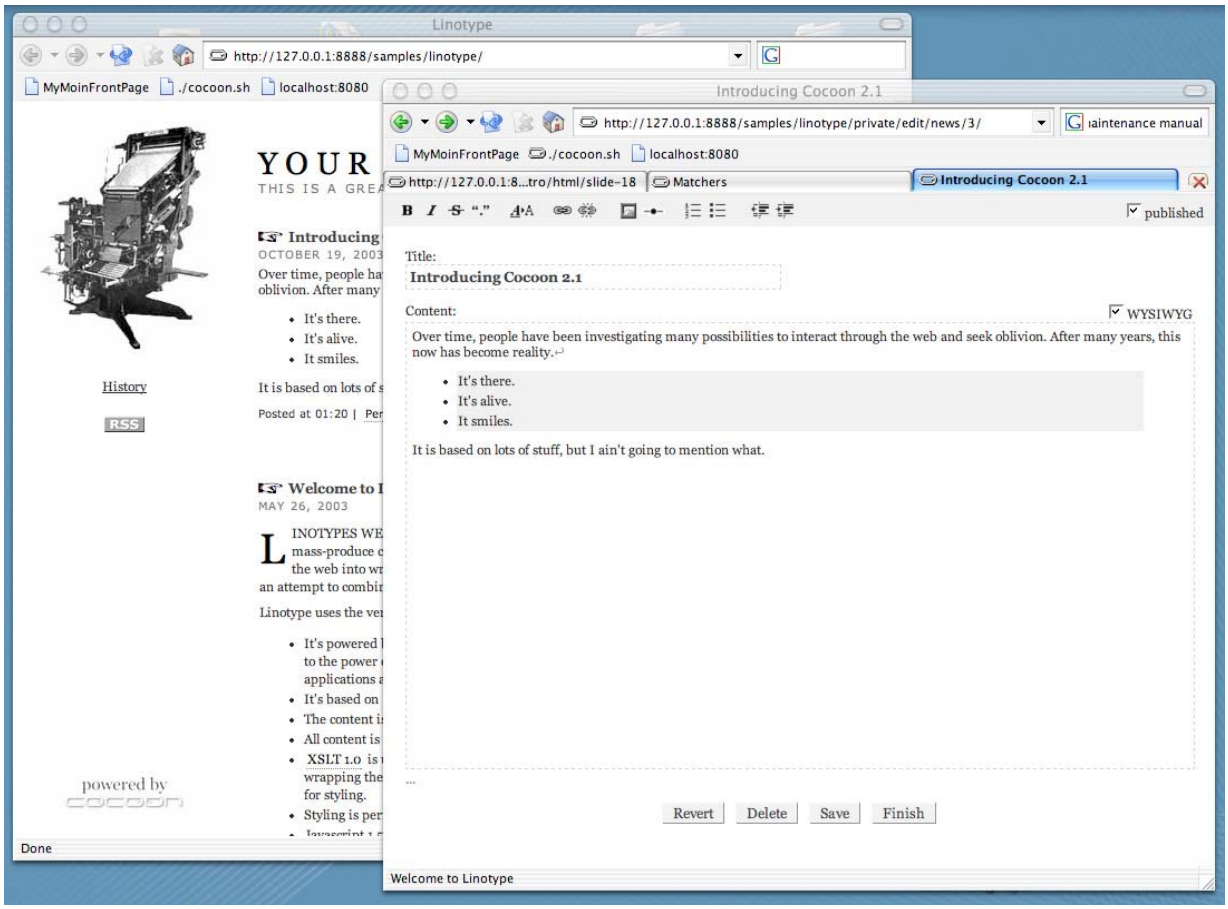
Databases

» RDBMS (SQLTransformer, ESQL logic sheets, DB Actions, preliminary flow support)
» Flow/OJB/Woody block
» XML Native DB: Xindice & eXist

Web Services

» Axis (SOAP client & server)

19. LINOTYPE



» Lightweight blogging tool
» Generic Flow example
» XHTML editing through Mozilla (Midas), IE support in the works

20. WOODY / COCOON FORMS
Official Cocoon form handling framework

Pragmatic approach to solid form handling.

Declarative - shouldn't require coding, even for complex forms.

Strongly typed.

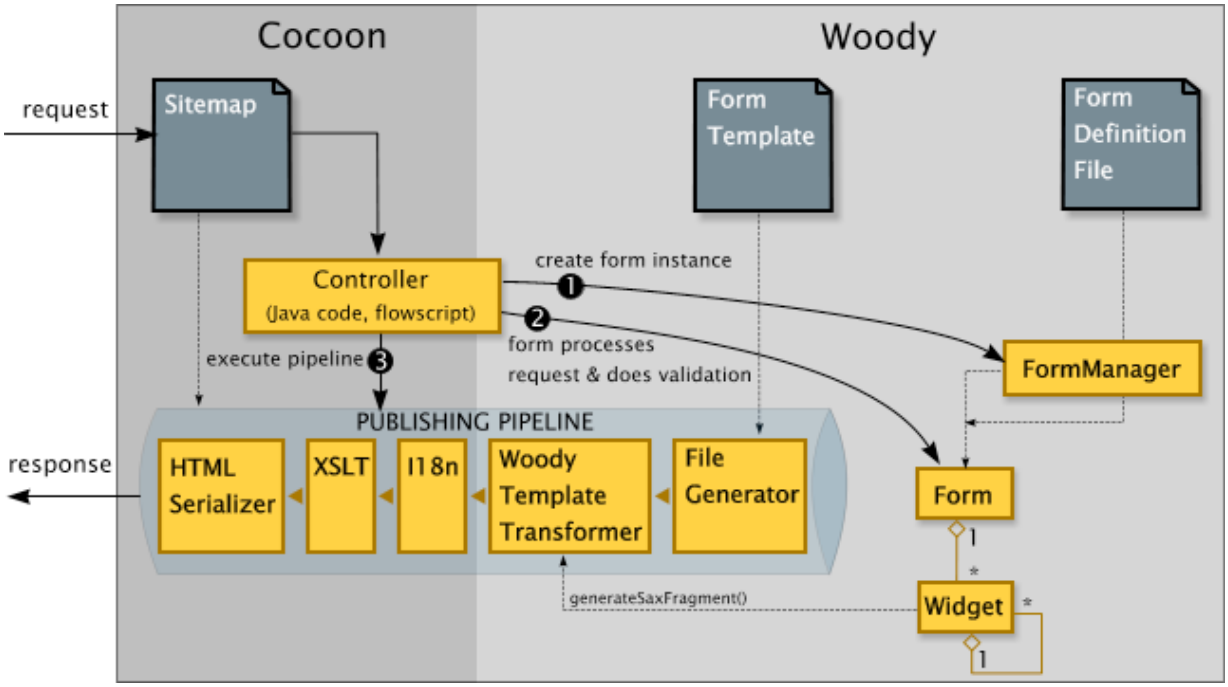
No(t much) fancy browser-side (Javascript) logic.

Consists of a Form definition, Controller, (opt.) a Binding component, and several approaches for displaying the form.

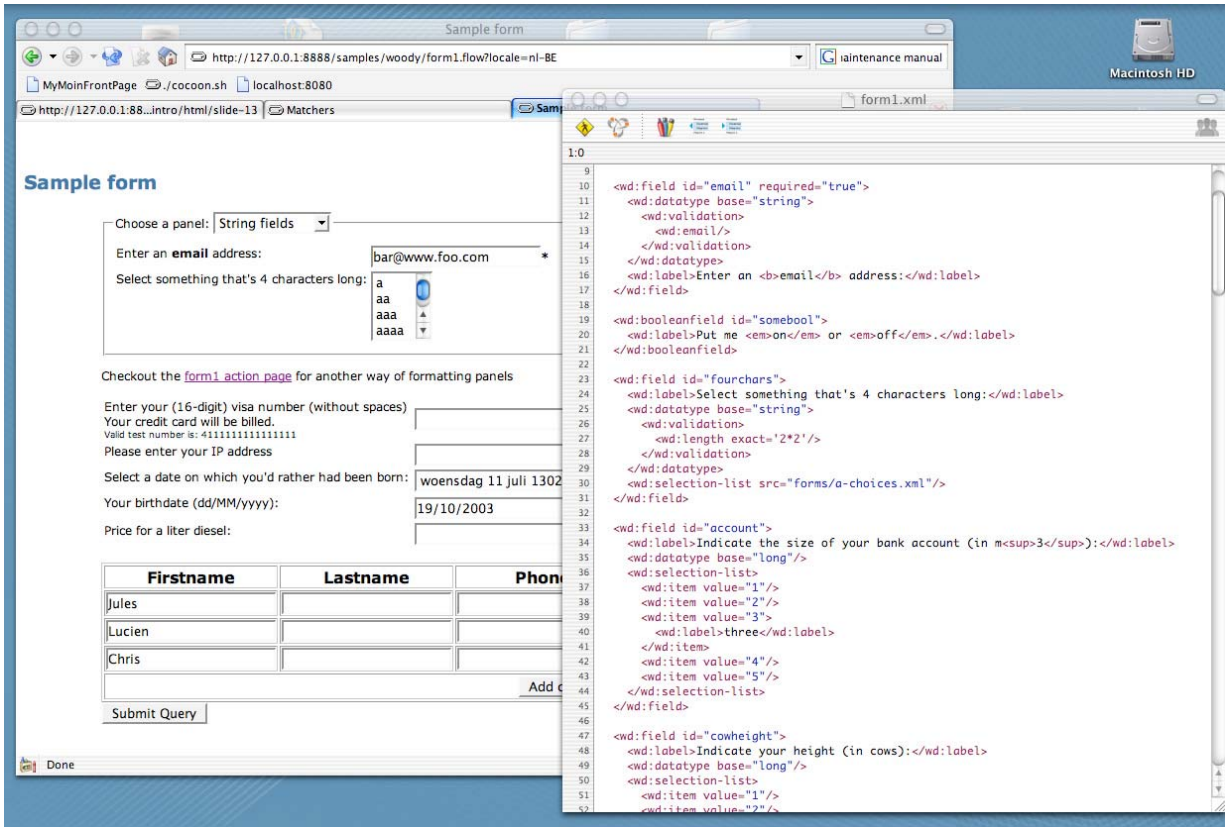
» WoodyTemplateTransformer
» JXTemplateGenerator/-Transformer

Orthogonal to application development choice (supports Actions, XSP, FlowScript ...)

21. WOODY OVERVIEW



22. WOODY EXAMPLE FORM



23. STUFF TO WATCH

Flow and webcontinuations

'Real Blocks'

24. COCOON AND OTHER FRAMEWORKS

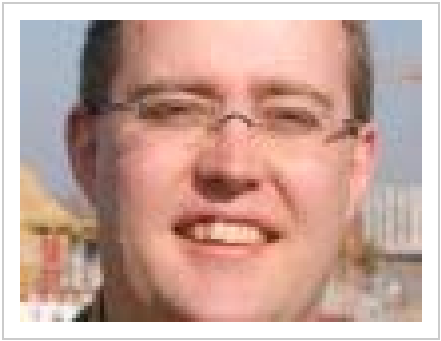
Cocoon and Struts

Cocoon and XForms

Cocoon and JetSpeed

25. About me

- » Apache Cocoon & Forrest committer
- » co-founder of Outerthought (http://outerthought.org/)
- » Chairman of XML BeLux user group
- » http://outerthought.org/whoweare/stevenn.html
- » http://wiki.cocoondev.org/Wiki.jsp?page=StevenNoels
- » http://blogs.cocoondev.org/stevenn/



26. Q&A

This presenation has ended.
Click anywhere to return.

_

